

Meshr-Lang — Documentation du langage

Version : 0.1.0

Statut : En conception active

Mainteneur : G. Prince - Architecte Principal **Dernière mise à jour :** 2025-01-12

Table des matières

Introduction

- Objectif du langage
 - Vision et philosophie
 - Objectifs principaux
 - Avantages clés

Syntaxe et concepts de base

- Syntaxe de base
 - Structure des fichiers
 - Style déclaratif
 - Commentaires
 - Types de valeurs primitives
 - Structure d'un module

Organisation du code

- Déclaration des modules et imports
 - Concept et objectifs
 - Architecture modulaire
 - Syntaxe
 - Sémantique
 - Types d'imports supportés
 - Ordre des déclarations
 - Export des déclarations
 - Convention d'arborescence

Métadonnées et annotations

- Annotations
 - Syntaxe
 - Règles
 - Annotations avec arguments typés
 - Déclaration des types d'annotations
 - Types de base autorisés
 - Contraintes sur les types String

- Annotations Standard (StdLib)

Types de données

- Énumérations
 - Concept et objectifs
 - Types d'énumérations
 - Syntaxe simple (inline)
 - Export groupé
 - Règles
 - Enums avec attributs typés
- Records et Collections
- Entités et Relations
- Traits
- Aspects

Contrôles avancés

- Modificateur `sealed`
- Artefacts définissables

Ressources et bonnes pratiques

- Décisions de conception
- Bibliothèque Standard (StdLib)
- À venir
- Bonnes pratiques et conventions

Annexes

- Annexes
 - Annexe A — Grammaire EBNF (référence)
 - Annexe B — Grammaire ANTLR (référence, Python target)
-

Objectif du langage

Meshr-Lang est un langage déclaratif spécialisé conçu pour décrire et modéliser les artefacts d'une architecture **Data-as-a-Product** et d'entreprise. Il permet de définir de manière structurée et exécutable les domaines métier, produits de données, contrats, aspects (métadonnées), équipes, politiques, et leurs interrelations.

Vision et philosophie

Meshr-Lang s'inscrit dans la philosophie du **Data Mesh** et des architectures d'entreprise modernes où :

- **La donnée est un produit** : Chaque dataset est traité comme un produit avec ses propres propriétaires, contrats, et SLA
- **La documentation est vivante** : Le code source devient la source de vérité pour l'architecture
- **L'automatisation est centrale** : Les modèles déclaratifs génèrent automatiquement les artefacts techniques
- **La gouvernance est intégrée** : Les politiques et contraintes sont exprimées directement dans le modèle

Objectifs principaux

Meshr-Lang vise à offrir une syntaxe lisible, formelle et exécutable pour :

Documentation vivante

- **Structurer la documentation** : Transformer la documentation statique en modèles exécutables
- **Maintenir la cohérence** : Assurer que la documentation reste synchronisée avec l'implémentation
- **Faciliter la compréhension** : Rendre l'architecture d'entreprise accessible à tous les acteurs

Génération automatique

- **Représentations techniques** : Générer automatiquement YAML, Rego, Terraform, OpenAPI, etc.
- **Code boilerplate** : Réduire la duplication et les erreurs de codage manuel
- **Validation continue** : Détecter les incohérences et violations de politiques

Gouvernance intégrée

- **Registres et catalogues** : Alimenter automatiquement les plateformes de gouvernance
- **Self-service** : Permettre aux équipes de découvrir et consommer les données facilement
- **Conformité** : Intégrer les exigences réglementaires (RGPD, SOX, etc.) dans le modèle

Avantages clés

- **Lisibilité** : Syntaxe claire et expressive inspirée des meilleures pratiques
- **Extensibilité** : Architecture modulaire permettant l'ajout de nouveaux concepts
- **Intégration** : Compatible avec l'écosystème moderne (CI/CD, IaC, observabilité)

- **Validation** : Vérification statique des modèles et détection d'erreurs précoces
-

Syntaxe de base

Meshr-lang est un langage de modélisation déclaratif conçu pour décrire des architectures d'entreprise, des produits, des domaines métier et leurs relations. Sa syntaxe s'inspire de langages modernes comme HCL (HashiCorp Configuration Language), Kotlin DSL, et GraphQL Schema Definition Language (SDL).

Structure des fichiers

Extension de fichier : `.meshr`

Tous les fichiers Meshr-lang utilisent l'extension `.meshr` et suivent une structure modulaire où chaque fichier représente un module autonome contenant des déclarations de types, d'entités, d'énumérations, et d'autres artefacts métier.

Style déclaratif

Meshr-lang adopte un style **déclaratif à blocs** qui privilégie la lisibilité et la structure hiérarchique :

- **Blocs imbriqués** : Les déclarations sont organisées en blocs logiques avec indentation
- **Syntaxe claire** : Utilisation de mots-clés explicites et de ponctuation minimale
- **Lisibilité** : Structure qui reflète naturellement la hiérarchie des concepts métier

Commentaires

Les commentaires permettent d'ajouter des explications et de la documentation directement dans le code source :

- **Commentaires de ligne** : Commencent par `//` et s'étendent jusqu'à la fin de la ligne
- **Commentaires de bloc** : Délimités par `/*` et `*/`, peuvent s'étendre sur plusieurs lignes
- **Ignorés par le parser** : Les commentaires ne font pas partie de l'AST (Abstract Syntax Tree)

Exemples de commentaires :

```
// Ceci est un commentaire de ligne
entity Customer {
  // Commentaire sur un champ
}
```

```

    name: String
}

/*
 * Ceci est un commentaire de bloc
 * qui peut s'étendre sur plusieurs lignes
 * pour documenter des concepts complexes
 */

```

Types de valeurs primitives

Meshr-lang supporte plusieurs types de valeurs primitives pour exprimer des données de base :

Chaînes de caractères

- **Syntaxe** : "texte entre guillemets doubles"
- **Usage** : Noms, descriptions, identifiants, valeurs textuelles
- **Exemples** : "Customer", "John Doe", "production"

Valeurs booléennes

- **Syntaxe** : true ou false
- **Usage** : Flags, conditions, propriétés binaires
- **Exemples** : isActive: true, isPublic: false

Nombres

- **Nombres décimaux** : 42, 3.14, -15.5
- **Nombres hexadécimaux** : 0xFF, 0x00FF00, 0x1A2B3C
- **Usage** : Compteurs, identifiants numériques, valeurs de configuration

Noms qualifiés

- **Syntaxe** : module.Type.EnumValue
- **Usage** : Références à des types, énumérations, ou constantes d'autres modules
- **Exemples** : privacy.Level.HIGH, core.Status.ACTIVE

Intervalles temporels

- **Syntaxe** : Interval <valeur> <unité>
- **Usage** : Durées, périodes, délais d'expiration
- **Exemples** :
 - Interval 12 month : 12 mois
 - Interval -1 day : -1 jour (hier)
 - Interval "2024-01" year to month : de janvier 2024 à maintenant

Structure d'un module

Un module Meshr-lang suit une structure hiérarchique bien définie :

1. **Déclarations d'import** (optionnelles)
2. **Déclaration du module** (obligatoire)
3. **Déclarations d'export** (optionnelles)
4. **Déclarations d'artefacts** (entités, énumérations, types, etc.)

Exemple de structure de module :

```
// Ceci est un commentaire simple

/*
  Ceci est un commentaire
  sur plusieurs lignes
*/
```

Déclaration des modules et imports

Le **module** est l'unité fondamentale d'organisation dans Meshr-lang. Il représente un espace de noms versionné et autonome qui contient des déclarations d'artefacts métier (entités, énumérations, types, etc.). Chaque fichier `.meshr` correspond à un module unique.

Concept et objectifs

Un module Meshr-lang sert à :

- **Encapsuler** : Regrouper des concepts métier liés dans un espace de noms cohérent
- **Versionner** : Permettre l'évolution des API et la gestion des dépendances
- **Réutiliser** : Faciliter l'import et la réutilisation de composants entre modules
- **Organiser** : Structurer l'architecture d'entreprise en composants modulaires
- **Isoler** : Définir des frontières claires entre différents domaines métier

Architecture modulaire

Meshr-lang encourage une architecture modulaire où :

- **Un module = un domaine métier** : Chaque module représente un domaine d'expertise spécifique
- **Dépendances explicites** : Les relations entre modules sont déclarées via des imports

- **Namespace hiérarchique** : Les noms de modules suivent une convention hiérarchique (ex: `core.types`, `marketing.analytics`)
- **Versioning sémantique** : Chaque module peut être versionné indépendamment

Syntaxe

```
@experimental
@version("0.2.0")
module marketing.analytics

// Import simple
import Customer from marketing.shared

// Import groupé (recommandé)
import { DataClassification, WithSecurity } from meshr.security
import { DataStewardship, WithStewardship } from meshr.governance
import { Version, Author, Documented } from meshr.annotations

// Import wildcard
import * from meshr.types

// autres déclarations
```

Sémantique

- Un fichier `.meshr` = un module unique.
- Le module définit le **namespace** des artefacts qu'il contient.
- Les `import` permettent d'accéder à des symboles publics d'autres modules.
- Des **annotations peuvent précéder** la déclaration `module` (ex.: `@experimental`, `@version("...")`).

Types d'imports supportés

Import simple

```
import Customer from marketing.shared
```

Importe un seul élément (enum, aspect, trait, etc.) d'un module.

Import groupé (recommandé)

```
import { DataClassification, WithSecurity } from meshr.security
```

Importe plusieurs éléments spécifiques d'un module. C'est la méthode recommandée car elle est explicite et évite les conflits de noms.

Import wildcard

```
import * from meshr.types
```

Importe tous les éléments exportés d'un module. Utilisez avec précaution pour éviter les conflits de noms.

Ordre des déclarations

L'ordre correct dans un module Meshr-Lang est :

```
module mon.module
```

```
// 1. Imports (après la déclaration du module)
import { Item1, Item2 } from autre.module
```

```
// 2. Exports (après les imports)
export { MonItem }
```

```
// 3. Déclarations (enums, aspects, traits, entités, etc.)
enum MonEnum is ("value1", "value2")
end
```

- Les `import` permettent d'accéder à des symboles **explicitement exportés** d'autres modules.
- Les éléments d'un module **ne sont pas visibles depuis l'extérieur** s'ils ne sont pas précédés du mot-clé `export`.
- L'import de `{ * }` signifie **importer tous les symboles exportés** du module cible.
- L'import de `{ A, B }` permet une **importation sélective**.
- Les accolades `{ }` sont **optionnelles uniquement si un seul identifiant** est importé.
- Si plusieurs identifiants sont importés d'un même module, l'usage des accolades est **obligatoire**.

Export des déclarations

Deux formes d'export sont possibles dans un module :

Export inline (immédiat)

```
export enum SensitivityLevel is (public, internal, restricted)
```

- La déclaration est rendue publique immédiatement.
- Facile à lire mais répétitif si plusieurs artefacts sont à exporter.

Export groupé (centralisé)


```
export { PIICategory, SensitivityLevel }
```

```
enum PIICategory is (contact, identity, financial, health, biometric, location)
enum SensitivityLevel is (public, internal, restricted, confidential)
```

- Doit apparaître juste **après les imports**, jamais en fin de module.
- Permet de déclarer tous les artefacts exportés en un seul point.
- Aucune déclaration listée dans ce bloc n'a besoin du mot-clé **export** en ligne.

Règles de priorité d'export

1. Une déclaration précédée de **export** est **immédiatement publique**.
2. Une déclaration listée dans **export { ... }** devient **publique à posteriori**.
3. Un même nom peut apparaître dans les deux formes (toléré).
4. Toute déclaration **non exportée explicitement** est **privée** au module.
 - Un fichier **.meshr** = un module unique.
 - Le module définit le **namespace** des artefacts qu'il contient.
 - Les **import** permettent d'accéder à des symboles publics d'autres modules.
 - Les annotations **@module_*** sont optionnelles mais encouragées pour documenter les modules.
 - Les **import** permettent d'accéder à des symboles **explicitement exportés** d'autres modules.
 - Les éléments d'un module **ne sont pas visibles depuis l'extérieur** s'ils ne sont pas précédés du mot-clé **export**.
 - L'import de **{ * }** signifie **importer tous les symboles exportés** du module cible.
 - L'import de **{ A, B }** permet une **importation sélective**.
 - Les accolades **{ }** sont **optionnelles uniquement si un seul identifiant** est importé.
 - Si plusieurs identifiants sont importés d'un même module, l'usage des accolades est **obligatoire**.

Convention d'arborescence

Par convention (non obligatoire), l'arborescence des fichiers reflète les noms qualifiés des modules :

Module	Fichier
core.types	modules/core/types.meshr
marketing.analytics	modules/marketing/analytics.meshr

Annotations

Les **annotations** permettent d'ajouter des métadonnées structurées sur n'importe quelle déclaration du langage Meshr (**module**, **product**, **domain**, **enum**, etc.). Elles sont inspirées de langages comme Java, Kotlin ou GraphQL.

Syntaxe

- Une annotation commence par @ suivie de son nom.
- Elle peut recevoir :
 - Aucun argument : `@experimental`
 - Une seule valeur : `@since("1.2.0")`
 - Une ou plusieurs paires clé-valeur : `@author(name="Greg")`
 - Des arguments mixtes : `@scope(level=ScopeLevel.Internal)`
- Plusieurs annotations peuvent être empilées au-dessus d'une déclaration.

Exemples

```
@experimental
@version("0.2.0")
@scope(level=ScopeLevel.Internal)
@author(
    name="Greg",
    email="greg@example.com"
)
@deprecated(reason="Use new module instead")
@confidentiality(privacy.Level.HIGH)
module metadata.annotations
```

Règles

- Les valeurs peuvent être :
 - des chaînes : `"texte"`
 - des identifiants : `Internal`
 - des chemins qualifiés : `privacy.Level.HIGH`
- Les paires **clé=valeur** peuvent être combinées dans une annotation.
- Les annotations sont **optionnelles** et **non normatives** mais peuvent être exploitées par les outils CLI ou LSP.

[2025-09-10] — Annotations avec arguments typés

- **Contexte** : Besoin d'enrichir les artefacts avec des métadonnées structurées.
- **Décision** : Support des annotations avec arguments (valeur unique ou paires clé/valeur), et valeurs typées (`string`, `boolean`, `number` — y compris `hex` —, `qualifiedName`, `interval`).

- **Pourquoi** : Permet une expressivité maximale tout en restant compatible avec une grammaire ANTLR claire et extensible.

Déclaration des types d'annotations

Meshr permet de déclarer des **types d'annotations** personnalisés à l'aide du mot-clé `annotation`.

Syntaxe

```
@Target(annotation)
@Retention(model)
annotation Documented is
    required summary : String
    optional description : String = ""
    optional deprecated : Boolean = false
end
```

Sémantique

- Les types d'annotations sont eux-mêmes annotables (`@Target`, `@Retention`, `@Repeatable`...).
- Le bloc interne suit la forme :
 - **required** : la valeur doit obligatoirement être fournie à l'usage, **aucune valeur par défaut autorisée**.
 - **optional** : la valeur est optionnelle, et peut être associée à une valeur par défaut.
- Si une annotation typée est utilisée sans renseigner un champ **optional**, alors la valeur par défaut s'applique.
- Les types autorisés incluent les **types de base** (`String`, `Boolean`, `Integer`, `Float`, `Double`, `Date`, `Datetime`, `Time`, `Timestamp`, `Geography`, `Bytes`, `Json`, `Interval`, `Range`) et les **noms qualifiés** (enums, types importés).
- Le mot-clé `end` est requis pour clore la déclaration.

Règle sémantique **required** vs **optional**

- **required** : champ **obligatoire**, **sans valeur par défaut**, doit être explicitement renseigné à l'usage.
- **optional** : champ **optionnel**, **avec ou sans valeur par défaut**.
 - Si valeur par défaut spécifiée → utilisée si champ non renseigné.
 - Sinon → champ absent à l'usage.

Types de base autorisés pour les annotations

Les types suivants peuvent être utilisés dans les déclarations d'annotations :

- `Boolean` : valeurs `true` ou `false`

- **String** : chaînes de caractères délimitées par des guillemets
- **Integer** : nombre entier (ex: 42)
- **Float** : nombre décimal (ex: 3.14)
- **Double** : précision étendue (équivalent sémantique à **Float** pour l'instant)
- **Date**, **Datetime**, **Time**, **Timestamp** : types temporels
- **Geography** : localisation géographique (future extension)
- **Bytes** : données binaires
- **Json** : valeurs encodées en JSON (future extension)
- **Interval** : intervalle sur un type temporel (ex: **Interval 2 hour**)
- **Range** : intervalle contigu entre deux valeurs ordonnées (ex: **Range 1..10**)

Contraintes sur les types String

Les types **String** peuvent être enrichis avec des contraintes pour valider le format et la longueur des chaînes :

Syntaxe des contraintes

```
// Contrainte de pattern (regex)
email : String pattern "[^@]+@[^@]+$"

// Contrainte de longueur
username : String length 3..20

// Contraintes combinées
phone : String pattern "[0-9]{10,15}$" length 10..15
```

Types de contraintes

- **pattern** : expression régulière pour valider le format
 - Exemple email : **String pattern "[^@]+@[^@]+\$"**
 - Exemple téléphone : **String pattern "[0-9]{10,15}\$"**
 - Exemple URL : **String pattern "https?://.*"**
- **length** : bornes sur la longueur de la chaîne
 - Longueur fixe : **String length 10..10**
 - Longueur variable : **String length 1..100**
 - Longueur minimale : **String length 5..**

Utilisation dans tous les contextes Les contraintes String peuvent être utilisées partout où un type **String** est autorisé :

```
// Dans les annotations
annotation UserProfile is
  required email : String pattern "[^@]+@[^@]+$"
  required username : String length 3..20
  optional phone : String pattern "[0-9]{10,15}$" length 10..15
end
```

```
// Dans les enums
enum HttpStatus(code: Integer, message: String pattern "[A-Z][a-z ]+$") is (
    ok(code = 200, message = "OK"),
    not_found(code = 404, message = "Not Found")
)

// Dans les records
record Contact is
    name : String length 1..100
    email : String pattern "^[^@]+@[^@]+$"
    phone : String pattern "[0-9]{10,15}$"
end

// Dans les traits
trait Identifiable is
    id : String pattern "[a-zA-Z0-9_-]+$" length 1..50
    name : String length 1..100
end
```

Règles sémantiques

- Les contraintes **pattern** et **length** peuvent être combinées
- L'ordre des contraintes n'est pas significatif
- Les contraintes sont optionnelles : **String** sans contrainte est valide
- Les erreurs de contraintes (double pattern, length incompatible) sont détectées par l'analyseur sémantique

Les annotations peuvent utiliser les valeurs suivantes:

- Chaînes: "abc"
- Booléens: **true**, **false**
- Numériques: 42, 3.14, 0xFF, 0x00FF00
- Noms qualifiés: **privacy.Level.HIGH**
- Intervalles: voir section dédiée ci-dessous

Les énumérations peuvent également être utilisées comme type d'attribut.

```
annotation Repeatable is
    optional value : Boolean = true
end

enum Color(value: Integer) is (
    red(value = 0xFF0000),
    green(value = 0x00FF00),
    blue(value = 0x0000FF)
)
```

Exemples

```
// Type d'annotation avec plusieurs champs
@Target(annotation)
@Retention(model)
annotation Example is
    required name : String
    optional version : String = "1.0"
    optional experimental : Boolean = false
end

// Annotation simple booléenne
@Target(annotation)
@Retention(model)
annotation Repeatable is
    optional value : Boolean = true
end
```

Tests valides

```
@Example(name="MeshrLang")
annotation Test1 is end

@Example(name="MeshrLang", experimental=true)
annotation Test2 is end
```

Tests invalides Les tests invalides sont implémentés dans le fichier `invalid-annotation-decl-test.meshr` et couvrent :

- **Erreurs de syntaxe dans les annotations** : Arguments d'annotations mal formés
- **Champs required non renseignés** : Validation des champs obligatoires
- **Redondance entre champs** : Détection des déclarations en double
- **Mauvais types** : Validation des types dans les annotations
- **Syntaxe invalide** : Structures d'annotations mal formées

Exemple d'erreur détectée :

```
@Target(annotation) // Erreur : argument mal formé
@Retention(model)
annotation InvalidAnnotation is
    optional : String = "missingName"
end
```

Annotations Standard (StdLib)

La bibliothèque standard Meshr-Lang fournit un ensemble d'annotations prédéfinies dans le module `meshr.annotations`, incluant les méta-annotations et les annotations communes.

Import des annotations

```
module mon.module

// Import des annotations standard
import {
  Target, Retention, Repeatable,
  Experimental, Deprecated, Version, Author,
  Scope, Confidentiality, Documented
} from meshr.annotations

export { MonEntite }
```

Méta-annotations

@Target Spécifie sur quels types d'éléments l'annotation peut être utilisée.

```
@Target("all")           // Tous les éléments
@Target("annotation")     // Seulement les annotations
@Target("entity")         // Seulement les entités
@Target("module")         // Seulement les modules
```

Valeurs possibles : - "all" : Tous les éléments - "annotation" : Annotations
- "enum" : Énumérations - "module" : Modules - "record" : Records - "trait"
: Traits - "aspect" : Aspects - "entity" : Entités - "type_relation" : Types
de relations - "relation" : Relations

@Retention Spécifie la durée de vie de l'annotation.

```
@Retention(compile) // Supprimée à la compilation
@Retention(model)   // Conservée dans le modèle
@Retention(export)  // Exportée avec l'artefact
```

@Repeatable Indique si l'annotation peut être utilisée plusieurs fois sur le même élément.

```
@Repeatable(true)  // Peut être répétée
@Repeatable(false) // Ne peut pas être répétée
```

Annotations communes

@Experimental Marque un élément comme expérimental.

```
@Experimental(reason="Feature is experimental and may change")
```

@Deprecated Marque un élément comme déprécié.

```
@Deprecated(  
    reason="Use new OrderV2 entity instead",  
    since="2024-01-01",  
    replacement="OrderV2"  
)
```

@Version Spécifie la version d'un élément.

```
@Version("1.2.0")
```

@Author Spécifie l'auteur d'un élément.

```
@Author(  
    name="Data Team",  
    email="data@company.com",  
    organization="ACME Corp"  
)
```

@Scope Spécifie la portée d'un élément.

```
@Scope(level="internal", description="Internal use only")
```

Niveaux possibles : - "public" : Public - "internal" : Interne - "private"
: Privé

@Confidentiality Spécifie le niveau de confidentialité.

```
@Confidentiality(  
    level="confidential",  
    classification="PII",  
    handling_instructions="Encrypt at rest"  
)
```

Niveaux possibles : - "public" : Public - "internal" : Interne -
"confidential" : Confidentiel - "restricted" : Restreint

@Documented Ajoute de la documentation structurée.

```
@Documented(  
    summary="Customer entity with personal information",  
    description="Core entity for customer data management",  
    examples="See examples/ directory",  
    see_also="CustomerV2 entity"  
)
```

Exemples d'utilisation

Module annoté

```
@Version("1.0.0")
@Author(name="Data Team", email="data@company.com")
@Scope(level="internal")
@Confidentiality(level="confidential")
@Documented(summary="Customer management module")
module examples.customer
```

Entité annotée

```
@Version("1.2.0")
@Author(name="Product Team")
@Documented(summary="Customer entity with personal information")
@Experimental(reason="New customer model, API may change")
entity Customer is
    id : String
    name : String
end
```

Annotation personnalisée

```
@Target("all")
@Retention(model)
@Repeatable(true)
annotation QualityGate is
    required level : String
    optional automated : Boolean = true
    optional reviewer : String = ""
end

@QualityGate(level="high", reviewer="senior-dev")
entity CriticalEntity is
    id : String
end
```

Intégration avec la stdlib Les annotations s'intègrent parfaitement avec les autres modules de la stdlib :

```
import { DataClassification, WithSecurity } from meshr.security
import { Version, Author, Documented } from meshr.annotations

@Version("1.0.0")
@Author(name="Security Team")
@Documented(summary="Secure customer entity")
entity SecureCustomer with WithSecurity is
    id : String
```

```

aspects {
  DataClassification {
    level: "confidential",
    encryption_required: true
  }
}
end

```

Énumérations

Les **énumérations** (ou enums) sont des types de données qui permettent de définir un ensemble fini et ordonné de valeurs symboliques. Elles sont particulièrement utiles pour représenter des catégories, des statuts, des niveaux de priorité, ou tout autre concept métier qui peut être exprimé par un nombre limité d'options prédéfinies.

Concept et objectifs

Les énumérations servent à :

- **Limiter les choix** : Restreindre les valeurs possibles à un ensemble prédéfini et valide
- **Améliorer la lisibilité** : Remplacer des valeurs numériques ou textuelles par des noms explicites
- **Assurer la cohérence** : Garantir que seules des valeurs valides sont utilisées dans le système
- **Faciliter la maintenance** : Centraliser la définition des valeurs possibles
- **Documenter le domaine** : Exprimer explicitement les options disponibles dans un contexte métier

Types d'énumérations

Meshr-lang supporte deux formes d'énumérations :

Énumérations simples

- **Valeurs symboliques** : Liste de noms représentant les options possibles
- **Usage** : Statuts, catégories, niveaux de priorité
- **Exemple** : (active, inactive, pending)

Énumérations avec paramètres

- **Valeurs enrichies** : Chaque valeur peut avoir des propriétés associées
- **Usage** : Codes d'erreur avec messages, statuts avec métadonnées
- **Exemple** : active(code = 200, message = "OK")

Syntaxe simple (inline)

```
export enum PIICategory is (contact, identity, financial, health, biometric, location)
```

- Le mot-clé **enum** introduit une énumération nommée.
- L'utilisation de **export** rend l'énumération visible à l'extérieur du module.
- Les valeurs sont listées entre parenthèses, séparées par des virgules.
- Cette forme convient aux déclarations rapides et lisibles.

Export groupé

```
export { PIICategory }
```

```
enum PIICategory is (contact, identity, financial, health, biometric, location)
```

- L'énumération est déclarée sans **export**, puis rendue publique via un bloc `export { ... }`.
- Cette forme permet de centraliser tous les artefacts publics après les imports.

Règles

- Les valeurs d'enum peuvent être des **identifiants** ou des **chaînes** (ex: "export").
- Les identifiants de valeurs doivent être uniques et respectent la casse.
- Une énumération peut être annotée, comme tout autre artefact.
- Le nom de l'énumération doit être un identifiant valide.

[2025-09-10] — Enums avec attributs typés

- **Contexte** : Il est utile de donner aux énumérations des attributs associés à chaque valeur (ex. code couleur, libellé, gravité).
- **Décision** : On introduit une forme enrichie de déclaration d'**enum**, où chaque valeur peut porter une ou plusieurs **propriétés typées**, similaires aux `enum class` de Kotlin.
- **Pourquoi** : Cela augmente l'expressivité du langage, permet des relations croisées entre enums, et reste compatible avec les aspects et politiques du langage.

Syntaxe enrichie

```
enum Color(value: Integer) is (  
    red(value = 0xFF0000),  
    green(value = 0x00FF00),  
    blue(value = 0x0000FF)  
)
```

```
enum Severity(color: Color, severity: String = "none") is (
```

```

    none(color = Color.blue),
    medium(color = Color.green, severity = "medium"),
    high(color = Color.red, severity = "high")
)

```

Remarque : les valeurs entières peuvent être exprimées en hexadécimal (ex: 0xFF0000) pour représenter des couleurs ou des flags binaires.

Règles

- Une énumération peut définir une **signature d'attributs** dans (...) immédiatement après le nom.
- Chaque valeur de l'énumération doit fournir des arguments nommés (ex : `color = Color.red`).
- Les attributs peuvent avoir une **valeur par défaut**, comme `severity: String = "none"`.
- Les types d'attributs peuvent être des **types de base** (cf. liste) ou des **noms qualifiés** (ex.: autre enum).
- L'ordre des arguments dans les valeurs n'a pas besoin de suivre celui de la signature.

Les nombres peuvent être fournis en **hexadécimal** (0xFF0000) pour des codes couleur, et les **intervalles** sont supportés dans les valeurs d'annotation.

Intervalles (Interval)

Les littéraux d'intervalle sont supportés dans les annotations via la forme `Interval ....`

Formes supportées:

- Partie simple: `Interval 12 month`, `Interval -1 day`, `Interval 2 hour`
- Avec bornes textuelles + précision: `Interval "2024-01" year to month`
- Précisions supportées: `year`, `quarter`, `month`, `week`, `day`, `hour`, `minute`, `second`, `millisecond`, `microsecond`

Notes:

- Le signe - est autorisé devant les nombres d'intervalle: `Interval -10 day`.
- Les nombres hexadécimaux sont acceptés: `Interval 0x10 second`.

Types composites: List, Map, Range, Record

Meshr supporte des types composites utilisables dans les signatures d'énumérations et les déclarations d'annotations via `typeRef`:

- List of T
- Map of K to V
- Range of T
- record référencé par nom (voir ci-dessous)

Exemples de types:

```
annotation Meta is
  required owner : String
  optional tags : List of String
  optional conf : Map of String to Integer
  optional span : Range of Integer
end
```

```
record Address is
  street : String
  zipcode : Integer
  extras : Map of String to Integer
end
```

Valeurs par défaut dans record Les champs d'un **record** peuvent recevoir une valeur par défaut:

```
record Address is
  street : String = "foo"
  zipcode : Integer = 42
  extras : Map of String to Integer = Map { "foo" : 42, "bar" : 69 }
end
```

Littéraux composites utilisables dans annotationValue et les valeurs d'enum

- List: List[1,2,3], List["a","b"]
- Map: Map{"k1":1, "k2":2}
- Record anonyme: Record{name:"Alice", age:42}
- Range: Range -2048 .. 2048, Range 1..10
- **Json**: Json{"key": "value"}, Json[1,2,3], Json{"\raw\":"\json\}"
- **Geography**: Geography"POINT(2.3522 48.8566)", Geography{"type":"Point","coordinates":[2.3522, 48.8566]}
- **Bytes**: Bytes"0xdeadbeef", Bytes[255, 0, 255], Bytes{"verified":false, "algorithm": "sha256"}

Exemple d'usage dans une enum:

```
enum Product(status: String, owners: List of String, address: Address, limits: Range of Integer)
  active(status = "ok", owners = List["ops","qa"], address = Record{street:"A", zipcode:75001}, limits = 1..10
)
```

Exemple d'usage avec des littéraux **Json**:

```

@ApiConfig(
    endpoint="https://api.example.com",
    headers=Json{"Content-Type": "application/json", "Authorization": "Bearer token"},
    schema=Json{"type": "object", "properties": {"name": {"type": "string"}}}
)
annotation ApiConfig is
    required endpoint : String
    optional headers : Json = Json{"Content-Type": "application/json"}
    optional schema : Json = Json{"type": "object"}
end

enum HttpStatus(code: Integer, details: Json) is (
    ok(code = 200, details = Json{"message": "Success", "data": {"count": 0}}),
    not_found(code = 404, details = Json{"error": "Not Found", "code": "E404"}),
    server_error(code = 500, details = Json{"error": "Internal Server Error", "stack": []})
)

```

Littéraux Bytes Les littéraux Bytes permettent de représenter des données binaires de trois façons :

- **Chaîne hexadécimale** : Bytes"0xdeadbeef", Bytes"FFD8FFFE0"
- **Tableau de valeurs numériques** : Bytes[255, 0, 255], Bytes[0x89, 0x50, 0x4E, 0x47]
- **Objet JSON** : Bytes{"verified": false, "algorithm": "sha256"}

Les valeurs dans les tableaux peuvent être des nombres entiers (positifs ou négatifs) ou des valeurs hexadécimales.

Exemple d'usage avec des littéraux **Bytes**:

```

@BinaryConfig(
    signature=Bytes"0xdeadbeef",
    hash=Bytes"a1b2c3d4e5f6",
    data=Bytes"U29tZSBiaW5hcnkgZGF0YQ=="
)
annotation BinaryConfig is
    required signature : Bytes
    optional hash : Bytes = Bytes"000000000000"
    optional data : Bytes = Bytes"SGVsbG8gV29ybGQ="
end

enum FileType(extension: String, magic_bytes: Bytes, header: Bytes) is (
    pdf(extension = "pdf", magic_bytes = Bytes"25504446", header = Bytes"255044462D"),
    jpeg(extension = "jpg", magic_bytes = Bytes"FFD8FFFE0", header = Bytes"FFD8FFFE000104A46494"),
    png(extension = "png", magic_bytes = Bytes"89504E47", header = Bytes"89504E470D0A1A0A")
)

```

```
record ApiResponse is
  status : Integer
  data : Json = Json{}
  metadata : Json = Json{"timestamp": "2024-01-01T00:00:00Z", "version": "1.0"}
end
```

Exemple d'usage avec des littéraux **Geography**:

```
@LocationConfig(
  office=Geography"POINT(2.3522 48.8566)",
  coverage_area=Geography"POLYGON((2.3522 48.8566, 2.3523 48.8567, 2.3524 48.8568, 2.3522 48.8566))",
  delivery_route=Geography"LINESTRING(2.3522 48.8566, 2.3523 48.8567, 2.3524 48.8568)"
)
annotation LocationConfig is
  required office : Geography
  optional coverage_area : Geography = Geography"POINT(0.0 0.0)"
  optional delivery_route : Geography = Geography"LINESTRING(0.0 0.0, 1.0 1.0)"
end

enum CityType(name: String, location: Geography, bounds: Geography) is (
  paris(name = "Paris", location = Geography"POINT(2.3522 48.8566)", bounds = Geography"POLYGON((2.3522 48.8566, 2.3523 48.8567, 2.3524 48.8568, 2.3522 48.8566))")
  london(name = "London", location = Geography"POINT(0.1276 51.5074)", bounds = Geography"POLYGON((0.1276 51.5074, 0.1277 51.5075, 0.1278 51.5076, 0.1276 51.5074))")
  tokyo(name = "Tokyo", location = Geography{"type": "Point", "coordinates": [139.6917, 35.6895]}, bounds = Geography"POLYGON((139.6917 35.6895, 139.6918 35.6896, 139.6919 35.6897, 139.6917 35.6895))")
)

record Store is
  name : String
  location : Geography
  service_area : Geography = Geography"POLYGON((0.0 0.0, 1.0 0.0, 1.0 1.0, 0.0 1.0, 0.0 0.0))"
  delivery_route : Geography = Geography"LINESTRING(0.0 0.0, 1.0 1.0)"
end
```

Exemples avancés (imbriqués)

```
record Contact is
  name : String
  emails : List of String
  attributes: Map of String to String = Map{"role": "owner"}
end

record Team is
  name : String
  members : List of Contact
  quotas : Map of String to Range of Integer = Map{"daily": Range -100 .. 100}
end

annotation Project is
```

```

    required name      : String
    optional team      : Team
    optional labels    : List of String = List["alpha","beta"]
    optional routing   : Map of String to Map of String to Integer
end

@Project(
  name = "mesh",
  team = Record{
    name: "core",
    members: List[
      Record{name:"Alice", emails: List["a@example.com"]},
      Record{name:"Bob",   emails: List["b@example.com","bob@work"]}
    ],
    quotas: Map{"daily": Range -50 .. 50}
  },
  routing = Map{
    "svcA": Map{"p": 80, "s": 20},
    "svcB": Map{"p": 60, "s": 40}
  }
)
module demo.advanced

```

Records et Collections

Les **Records** sont des structures de données composées qui permettent de regrouper des champs typés. Ils supportent la composition via les traits et peuvent contenir des **collections** (List, Map, Range).

Syntaxe des Records

```

record Contact is
  name : String
  email : String
  age : Integer = 0
end

// Record avec composition de traits
record User with Audit, Contactable is
  id : String
  role : String
end

```

Types de Collections

Meshr supporte quatre types de collections :

List - Listes typées

```
record UserProfile is
  tags : List of String = List["default", "user"]
  scores : List of Integer = List[85, 92, 78]
end
```

Map - Dictionnaires typés

```
record Configuration is
  settings : Map of String to String = Map{"theme": "dark", "lang": "fr"}
  limits : Map of String to Integer = Map{"cpu": 80, "memory": 512}
end
```

Range - Intervalles typés

```
record Metrics is
  score_range : Range of Integer = Range 0..100
  temperature : Range of Float = Range -40.0..60.0
end
```

Record anonyme - Structures temporaires

```
@UserConfig(
  profile = Record{name: "Alice", age: 30},
  preferences = Record{theme: "dark", notifications: true}
)
annotation UserConfig is
  required profile : UserProfile
  required preferences : Preferences
end
```

Collections imbriquées

Les collections peuvent être imbriquées pour créer des structures complexes :

```
record ComplexData is
  // List de Maps
  configs : List of Map of String to String = List[
    Map{"env": "dev", "debug": "true"},
    Map{"env": "prod", "debug": "false"}
  ]

  // Map de Lists
  categories : Map of String to List of String = Map{
    "colors": List["red", "green", "blue"],
    "sizes": List["small", "medium", "large"]
  }
}
```

```

// Map de Ranges
limits : Map of String to Range of Integer = Map{
  "cpu": Range 0..100,
  "memory": Range 0..80
}
end

```

Usage dans tous les contextes

Les collections peuvent être utilisées partout où un type est attendu :

Dans les annotations

```

@Project(
  tags = List["alpha", "beta"],
  config = Map{"timeout": 30, "retries": 3},
  range = Range 1..100
)
annotation Project is
  required tags : List of String
  required config : Map of String to Integer
  required range : Range of Integer
end

```

Dans les enums

```

enum Status(
  code: Integer,
  tags: List of String,
  metadata: Map of String to String
) is (
  active(
    code = 200,
    tags = List["ok", "healthy"],
    metadata = Map{"type": "success", "level": "info"}
  )
)

```

Dans les traits

```

trait WithMetadata is
  tags : List of String
  config : Map of String to String
  limits : Range of Integer
end

```

Dans les aspects

```
aspect DataQuality is
  metrics : List of String = List["accuracy", "completeness"]
  thresholds : Map of String to Float = Map{"accuracy": 0.95}
  score_range : Range of Integer = Range 0..100
end
```

Règles sémantiques

- Les collections sont **typées** : `List of String`, `Map of String to Integer`
 - Les valeurs par défaut utilisent les **littéraux** : `List["a", "b"]`, `Map{"k": "v"}`
 - Les collections peuvent être **imbriquées** : `List of Map of String to String`
 - Les **références circulaires** sont interdites
 - Les types dans les collections doivent être **définis** ou **importés**
-

Entités et Relations

Les **Entités** et **Relations** sont des artefacts déclaratifs pour la modélisation de données dans l'architecture Data-as-a-Product. Elles permettent de définir des structures de données persistantes et leurs interconnexions.

Entités

Une **entité** est un artefact fondamental de Meshr-lang qui représente une structure de données persistante dans l'architecture d'entreprise. Elle modélise les concepts métier centraux et leurs propriétés.

Concept et rôle Les entités servent à :

- **Modéliser les concepts métier** : Représenter les objets centraux du domaine (Customer, Product, Order, etc.)
- **Définir la structure des données** : Spécifier les champs, types, et contraintes
- **Assurer la cohérence** : Fournir un modèle de données unifié à travers l'organisation
- **Faciliter l'intégration** : Servir de contrat pour les APIs et les échanges de données
- **Documenter le domaine** : Exprimer explicitement les concepts et leurs relations

Caractéristiques des entités Une entité Meshr-lang possède :

- **Champs typés** : Chaque champ a un type explicite (String, Integer, Date, etc.)
- **Contraintes** : Validation des données via des patterns, ranges, ou contraintes personnalisées
- **Traits** : Comportements réutilisables appliqués à l'entité (audit, versioning, etc.)
- **Aspects** : Métadonnées et annotations spécifiques au contexte métier
- **Relations** : Connexions avec d'autres entités via des relations typées

Syntaxe des Entités

```
entity <Name> [with <TraitName> (, <TraitName>)*] is
  <field declarations>
  [aspects { <AspectInstance>* }]
end
```

Exemples d'Entités

```
// Entité simple
entity Customer with WithAudit is
  CustomerId : String
  Email : String pattern "^[^@]+@[^@]+$"
  BirthDate : Date
  Phone : String length 10..15
end

// Entité avec aspects
entity SensitiveData with WithAudit is
  DataId : String
  Content : String
  Classification : String

  aspects {
    DataRetention { retention: Interval 5 year, steward: "dpo@example.com" }
  }
end
```

Types de Relations

Un **type de relation** est un modèle réutilisable décrivant les propriétés, contraintes et traits qu'une relation peut hériter.

Syntaxe des Types de Relations

```
type relation <Name> [with <TraitName> (, <TraitName>)*] is
  <field declarations>
```

```

    [aspects { <AspectInstance>* }]
end

```

Exemples de Types de Relations

```

type relation PLACED with WithAudit is
    since : Date
    channel : Channel
    quantity : Integer = 1
end

```

```

type relation FRIENDSHIP with WithAudit is
    since : Date
    note : String length 0..280
    closeness : Integer = 50
end

```

Relations

Une **relation** relie deux entités. Elle peut être non typée (définie ad hoc) ou typée (basée sur un type de relation), et peut être unidirectionnelle ou bidirectionnelle.

Syntaxe des Relations

```

[bidirectional] relation <Name> [of type <RelationType>] [with <TraitName> (, <TraitName>)*]
    from <Entity>(<Field>[, <Field>]*)
    to   <Entity>(<Field>[, <Field>]*)
    <field declarations>
    [aspects { <AspectInstance>* }]
end

```

Exemples de Relations

```

// Relation non typée
relation CurrentAssignment is
    from Customer(SalesRepId)
    to   Employee(EmployeeId)

    since : Date
    note : String length 0..140
end

```

```

// Relation typée
relation ProductOrder of type PLACED is
    from Product(ProductId)
    to   Order(ProductId)

```

```

    discount : Float = 0.0
    special_instructions : String length 0..200
end

// Relation bidirectionnelle typée
bidirectional relation Friendship of type FRIENDSHIP is
    from Customer(CustomerId)
    to Customer(CustomerId)

    mutual_interests : List of String = List["shopping", "technology"]
end

```

Caractéristiques des Entités et Relations

- **Composition de traits** : Support de `with` pour injecter des comportements
 - **Aspects** : Instanciation d'aspects via `aspects { }`
 - **Modificateurs** : Support de `sealed` et `export`
 - **Annotations** : Métadonnées attachables via `@Annotation`
 - **Types de relations** : Réutilisation via `of type <RelationType>`
 - **Bidirectionnalité** : Relations symétriques via `bidirectional`
 - **Extrémités** : Définition des clés de jointure via `from/to`
-

Traits

Un **trait** est un mécanisme de composition et de réutilisation dans Meshrlang qui permet de définir des blocs de déclarations (champs, métadonnées, contraintes, aspects) qui peuvent être injectés dans d'autres artefacts via la clause `with`. Les traits favorisent la factorisation, la composition déclarative, et l'élimination de la duplication de code.

Concept et rôle

Les traits servent à :

- **Factoriser le code** : Éviter la duplication de déclarations communes entre artefacts
- **Composer des comportements** : Combiner plusieurs traits pour créer des artefacts complexes
- **Standardiser les patterns** : Définir des conventions et patterns réutilisables
- **Maintenir la cohérence** : Assurer que les mêmes propriétés sont définies de manière identique

- **Faciliter l'évolution** : Modifier un trait pour impacter tous les artefacts qui l'utilisent

Caractéristiques des traits

Un trait Meshr-lang possède :

- **Champs typés** : Définition de propriétés avec types et contraintes
- **Composition** : Possibilité d'utiliser d'autres traits via **with**
- **Aspects intégrés** : Instanciation d'aspects qui seront hérités
- **Injection** : Application aux artefacts via la clause **with**
- **Réutilisabilité** : Un même trait peut être utilisé par plusieurs artefacts

Différence avec les aspects

Traits	Aspects
Définissent des champs et comportements	Définissent des métadonnées et propriétés
Sont injectés via with	Sont instanciés via aspects { ... }
Ajoutent de la structure	Ajoutent du contexte
Héritage de champs	Héritage de métadonnées

Syntaxe

```
trait <Identifiant> [with Base1, Base2] is
  <FieldName> : typeRef [= <valeur>]
  ...
  [aspects { <AspectInstance>* }]
end
```

Injection via with

La clause **with** permet d'injecter un ou plusieurs traits dans un artefact. Aujourd'hui, elle est supportée sur **record** et **trait**.

```
trait Audit is
  CreatedAt : Timestamp
  UpdatedAt : Timestamp
end

record Contact with Audit is
  Id : String
end

trait Contact is
  Email : String
```

```

end

trait ExtendedContact with Contact is
  Phone : String
end

record Person with Contact, Audit is
  Name : String
end

```

Aspects dans les Traits

Les traits peuvent contenir des **instanciations d'aspects** qui seront héritées par tous les artefacts qui utilisent le trait via **with**. Cela permet de factoriser les politiques transversales.

```

// Définition des aspects
aspect DataRetention is
  retention : Interval
  steward : String
end

aspect SecurityLevel is
  level : String = "public"
  encryption_required : Boolean = false
end

// Trait avec aspects
trait WithSecurity is
  access_level : String
  permissions : List of String

  aspects {
    SecurityLevel { level: "restricted", encryption_required: true },
    DataRetention { retention: Interval 5 year, steward: "security@example.com" }
  }
end

// Utilisation du trait avec ses aspects
record User with WithSecurity is
  id : String
  name : String
end

```

Héritage des Aspects : - Les aspects définis dans un trait sont **automatiquement appliqués** aux artefacts qui utilisent ce trait - Les aspects peuvent être **redéfinis** au niveau de l'artefact consommateur si nécessaire - Les règles

de **composition des aspects** s'appliquent normalement (intersection, explicitation des conflits)

Règles de composition (sémantique)

- Monotonie: on ne relâche pas les exigences (required → optional interdit, suppression de contraintes interdite).
- Intersection: des contraintes compatibles s'additionnent; on garde la version la plus restrictive.
- Explicitation: en cas de conflit dur (types différents, contraintes incompatibles, aspects contradictoires), l'artefact consommateur doit redéfinir explicitement.

Récapitulatif:

Situation	Exemple	Résolution
Compatibilité parfaite	X: string + X: string	Fusion automatique
Extension compatible	Y: string + Y: string required	required l'emporte
Contraintes compatibles	Z: string + Z: string pattern "^[A-Z]+\$"	Conserve la contrainte (intersection)
Conflit (types)	A: int + A: string	Erreur → redéfinir dans l'artefact
Conflit (contraintes)	B: string length 1..10 + B: string length 20..30	Erreur → redéfinir
Conflit (aspects)	C aspects { PII{level:high} } + C aspects { PII{level:low} }	Erreur → redéfinir

Notes: - Ces règles sont vérifiées par les outils (validation sémantique), pas à l'analyse syntaxique. - **typeRef** est autorisé dans les champs de **trait**, de **record**, dans les attributs d'énum et dans les champs d'annotations.

[2025-09-09] — Module = unité, namespace et fichier unique

- **Contexte** : Il fallait décider si l'on autorisait plusieurs modules par fichier, et s'il y avait un lien fort avec l'arborescence.
- **Décision** : Le langage impose qu'un fichier **.meshr** déclare **un seul module**. Le nom du module est un **namespace**. L'arborescence n'est **pas contrainte**, mais une convention est proposée.

- **Pourquoi** : Cela permet une résolution simple par les outils (CLI, LSP), évite les collisions de symboles, et reste lisible.
-

Aspects

Un **Aspect** est un mécanisme de métadonnées structurées dans Meshr-lang qui permet d'enrichir et d'annoter les artefacts avec des informations contextuelles spécifiques au domaine métier. Contrairement aux traits qui définissent des comportements, les aspects se concentrent sur la description de propriétés, contraintes, et métadonnées.

Concept et rôle

Les aspects servent à :

- **Enrichir les métadonnées** : Ajouter des informations contextuelles aux artefacts (gouvernance, qualité, sécurité)
- **Définir des politiques** : Exprimer des règles métier et des contraintes de conformité
- **Documenter le contexte** : Capturer des informations sur l'origine, l'usage, et la signification des données
- **Faciliter la gouvernance** : Intégrer les exigences réglementaires et les bonnes pratiques
- **Améliorer la traçabilité** : Associer des informations de provenance et de lineage

Caractéristiques des aspects

Un aspect Meshr-lang possède :

- **Champs typés** : Propriétés avec types explicites et contraintes optionnelles
- **Valeurs par défaut** : Définition de valeurs par défaut pour les champs optionnels
- **Héritage** : Possibilité d'étendre d'autres aspects pour la réutilisation
- **Instanciation** : Application aux artefacts via des blocs `aspects { ... }`
- **Composition** : Combinaison de plusieurs aspects sur un même artefact

Syntaxe de base

```
// Aspect simple
aspect BusinessDesc is
  domain : String
  summary : String
end
```

```

// Aspect abstrait
abstract aspect GovernanceBase is
    steward    : String
    policy_id  : String
end

// Aspect avec héritage
aspect DataRetention extends GovernanceBase is
    retention  : Interval
end

// Aspect avec composition (traits)
aspect DataQuality extends GovernanceBase with WithLifecycle is
    quality_score : Float
    validation_rules : List of String
end

```

Héritage et composition

Les Aspects supportent : - **Héritage simple** : `extends AspectName` - **Composition** : `with TraitName` (même syntaxe que les records/traits) - **Combinaison** : `extends AspectName with Trait1, Trait2`

```

trait WithLifecycle is
    created_at : Timestamp
    updated_at : Timestamp
end

abstract aspect GovernanceBase with WithLifecycle is
    steward    : String
    policy_id  : String
end

aspect DataRetention extends GovernanceBase is
    retention  : Interval
end

```

Types d'Aspects

- **Aspects abstraits** : `abstract aspect` - définitions génériques non instanciables
- **Aspects concrets** : `aspect` - instanciables dans les artefacts

Champs et contraintes

Les Aspects supportent tous les types et contraintes disponibles :

```

aspect ComprehensiveMetadata is
  // Types de base avec contraintes
  name : String length 1..100
  email : String pattern "^[^@]+@[^@]+$"

  // Types composites
  tags : List of String = List["default"]
  metadata : Map of String to String = Map{"source":"manual"}
  range : Range of Integer = Range 0..100

  // Types temporels
  created : Timestamp
  duration : Interval

  // Valeurs par défaut
  active : Boolean = true
  count : Integer = 0
end

```

Annotations sur les Aspects

Les Aspects peuvent être annotés comme tous les autres artefacts :

```

@since("1.0.0")
@id("business-desc")
@display_name("Business Description")
@description("Un aspect permettant d'enrichir les champs avec un domaine métier et un résumé")
aspect BusinessDesc is
  domain : String
  summary : String
end

```

Règles de composition

Les mêmes règles que pour les Traits s'appliquent lors de la fusion de définitions :

- **Fusion automatique** si signatures identiques
- **Renforcement** si extension compatible (ex. optional $\rightarrow$ required)
- **Intersection** des contraintes si compatibles
- **Erreur explicite** si conflit strict (type, contrainte, aspect key clash)

Instanciation (future)

Les Aspects seront instanciés avec le mot-clé **aspects** { ... } au niveau d'un artefact ou d'un champ :

```
// Syntaxe future pour l'instanciation
entity Customer is
  CustomerId : String
  Email      : String
  aspects {
    DataRetention {
      retention : Interval 1 year,
      steward   : "dpo@example.com",
      policy_id : "GDPR-001"
    }
  }
end
```

Modificateur sealed

Le modificateur **sealed** permet de **verrouiller l'extension** d'un artefact. Par défaut, tous les artefacts sont **ouverts** (extensibles). Un artefact marqué **sealed** ne peut plus être hérité, étendu ou redéfini partiellement. L'usage par référence ou instanciation **reste autorisé**.

Syntaxe

Le modificateur **sealed** peut précéder : - **sealed enum** - **sealed record** - **sealed trait** - **sealed aspect**

Les **annotations** ne peuvent pas être **sealed**. Toute tentative doit produire une erreur.

Règles d'héritage et d'extension

Enum

```
sealed enum Color is ("red", "green", "blue")
```

- **INTERDIT** : étendre ou redéfinir **Color**
- **AUTORISÉ** : utiliser **Color** comme type de champ

Record

```
sealed record Address is
  street : String
  city   : String
end
```

- **INTERDIT** : hériter de **Address**
- **AUTORISÉ** : l'utiliser comme type

Trait

```
sealed trait WithAudit is
  created_at : Timestamp
  updated_at : Timestamp
end
```

- **INTERDIT** : créer trait `ExtendedAudit` extends `WithAudit`
- **AUTORISÉ** : utiliser `WithAudit` via `with`

Aspect

```
sealed aspect Governance is
  steward   : String
  policy_id : String
end
```

- **INTERDIT** : aspect `AdvancedGovernance` extends `Governance`
- **AUTORISÉ** : instancier `Governance` dans un bloc `aspects { ... }`

Exemples valides

```
sealed enum Color is ("red", "green", "blue")
```

```
sealed record Address is
  street : String
  city   : String
end
```

```
sealed trait WithAudit is
  created_at : Timestamp
  updated_at : Timestamp
end
```

```
sealed aspect Governance is
  steward   : String
  policy_id : String
end
```

```
record User is
  name      : String
  address   : Address // Usage autorisé d'un record sealed
  color     : Color   // Usage autorisé d'un enum sealed
end
```

```
record AuditLog with WithAudit is // Usage autorisé d'un trait sealed
  action : String
end
```

Exemples invalides

```
//  enum scellée ne peut pas être étendue
enum ExtendedColor extends Color is ("yellow")

//  record scellé ne peut pas être hérité
record FullAddress extends Address is
  country : String
end

//  trait scellé ne peut pas être hérité
trait ExtendedAudit extends WithAudit is
  deleted_at : Timestamp
end

//  aspect scellé ne peut pas être hérité
aspect AdvancedGovernance extends Governance is
  extra : String
end

//  annotation ne peut pas être sealed
sealed annotation InvalidAnnotation is
  required name : String
end
```

Règles sémantiques

- Un artefact **sealed** ne peut pas être étendu via **extends**
 - Un artefact **sealed** peut être utilisé comme type de référence
 - Un trait **sealed** peut être composé via **with**
 - Un aspect **sealed** peut être instancié dans des blocs **aspects**
 - Les **annotations** ne peuvent jamais être **sealed**
 - L'ordre des modificateurs est : **sealed** **abstract** **aspect** (pas **abstract sealed**)
-

Artefacts définissables

1. domain

Déclaration d'un domaine hiérarchique.

```
domain retail.marketing {
  description = "Marketing domain for the retail perimeter"
}
```

2. product

Déclaration d'un produit de données.

```
product leads_b2c_import {  
  domain = retail.marketing  
  contracts = [  
    contract:marketing:leads_b2c_import:1.0.0  
  ]  
}
```

3. contract

(Défini dans une autre couche ou généré via DCDL)

4. aspect

(TBD)

5. team

(TBD)

6. policy

(TBD)

Décisions de conception

[2025-09-09] — Syntaxe déclarative avec blocs `{}`

- **Contexte** : Le langage doit rester simple, lisible pour les métiers, mais formalisable en grammaire ANTLR.
- **Décision** : Utiliser une syntaxe à blocs (`{}`), proche de HCL/Terraform.
- **Pourquoi** : Meilleure lisibilité et extensibilité que YAML/JSON, tout en étant facilement parsable.

[2025-09-10] — Visibilité par export explicite

- **Contexte** : Le langage devait gérer la visibilité entre modules pour encourager l'encapsulation.
- **Décision** : Un artefact déclaré dans un module n'est accessible depuis l'extérieur que s'il est précédé du mot-clé **export**.
- **Pourquoi** : Cette règle encourage une séparation claire entre API publique et éléments internes, facilite la documentation automatique et le linting.

[2025-09-10] — Deux formes d'export : inline ou groupé

- **Contexte** : Besoin de contrôler la visibilité des artefacts à l'extérieur du module.
- **Décision** : Deux formes sont supportées : inline (`export decl`) et groupée (`export { A, B }`).
- **Justification** : Favorise à la fois la lisibilité locale (inline) et la gestion explicite de l'API publique (groupé après les imports).

[2025-09-12] — Modificateur `sealed` pour contrôler l'extensibilité

- **Contexte** : Besoin de verrouiller l'extension de certains artefacts pour garantir la stabilité de l'API et éviter les dérivations non contrôlées.
- **Décision** : Introduction du modificateur `sealed` applicable aux `enum`, `record`, `trait` et `aspect`. Les annotations ne peuvent pas être sealed.
- **Pourquoi** : Permet de définir des contrats stables (ex: enums de statut, records d'adresse) tout en conservant la flexibilité de composition via `with` pour les traits.

[2025-09-12] — Types de collections typés et littéraux

- **Contexte** : Besoin de structures de données complexes pour représenter des métadonnées, configurations et données structurées dans les artefacts Data Mesh.
- **Décision** : Introduction de quatre types de collections : `List of T`, `Map of K to V`, `Range of T`, et `Record` anonyme, avec des littéraux syntaxiques (`List[...]`, `Map{...}`, `Range a..b`, `Record{...}`).
- **Pourquoi** : Permet une expressivité maximale pour les métadonnées complexes tout en maintenant la sécurité des types et la lisibilité du code. Les collections peuvent être imbriquées et utilisées dans tous les contextes (annotations, enums, records, traits, aspects).

[2025-09-12] — Entités et Relations pour la modélisation de données

- **Contexte** : Besoin de modéliser des structures de données persistantes et leurs interconnexions dans l'architecture Data-as-a-Product, avec support des relations complexes et de la gouvernance des données.
- **Décision** : Introduction de trois nouveaux artefacts : `entity` (structures de données persistantes), `type relation` (modèles réutilisables de relations), et `relation` (connexions entre entités avec support bidirectionnel).
- **Pourquoi** : Permet une modélisation complète du domaine métier avec support des relations complexes, de la gouvernance via les aspects, et de la réutilisabilité via les types de relations. Les entités et relations supportent tous les modificateurs existants (`sealed`, `export`, annotations) et s'intègrent parfaitement avec le système de traits et d'aspects.

Bibliothèque Standard (StdLib)

La **bibliothèque standard** de Meshr-Lang fournit un ensemble d'artefacts prédéfinis, d'aspects et de types communs pour faciliter le développement d'architectures Data-as-a-Product. Elle est organisée en modules thématiques et est disponible dans le dossier `stdlib/`.

Objectifs

- **Réutilisabilité** : Composants standards pour les cas d'usage courants
- **Cohérence** : Conventions et patterns établis
- **Productivité** : Réduction du temps de développement
- **Qualité** : Composants testés et validés

Modules disponibles

`meshr.security`

```
// Aspects de sécurité
aspect DataClassification is
  level : String // "public", "internal", "confidential", "restricted"
  encryption_required : Boolean = false
  access_control : String = "role-based"
end
```

```
aspect GDPRCompliance is
  data_subject_rights : List of String
  retention_period : Interval
  lawful_basis : String
  dpo_contact : String
end
```

```
// Types de relations de sécurité
type relation ACCESS_CONTROL is
  granted_by : String
  granted_at : Timestamp
  expires_at : Timestamp
  permissions : List of String
end
```

`meshr.governance`

```
// Aspects de gouvernance
aspect DataStewardship is
  steward : String
  owner : String
  business_domain : String
  last_review : Date
```

```

end

aspect DataQuality is
  quality_score : Float range 0.0..1.0
  validation_rules : List of String
  monitoring_frequency : Interval
  alert_threshold : Float
end

// Traits de gouvernance
trait WithStewardship is
  steward : String
  owner : String

  aspects {
    DataStewardship { steward: steward, owner: owner, business_domain: "default" }
  }
end

meshr.business

// Types métier communs
enum BusinessDomain is (
  "finance",
  "marketing",
  "sales",
  "hr",
  "operations",
  "product"
)

enum DataSensitivity is (
  "public",
  "internal",
  "confidential",
  "restricted"
)

// Traits métier
trait WithBusinessContext is
  domain : BusinessDomain
  sensitivity : DataSensitivity
  business_owner : String
end

meshr.lifecycle

```

```

// Aspects de cycle de vie
aspect DataLifecycle is
  created_at : Timestamp
  updated_at : Timestamp
  version : String
  status : String // "draft", "active", "deprecated", "archived"
  deprecation_date : Date
end

// Traits de cycle de vie
trait WithLifecycle is
  created_at : Timestamp
  updated_at : Timestamp
  version : String = "1.0.0"

  aspects {
    DataLifecycle {
      created_at: created_at,
      updated_at: updated_at,
      version: version,
      status: "active"
    }
  }
end

meshr.annotations

// Méta-annotations
@Target("annotation")
@Retention(model)
annotation Target is
  required value : AnnotationTarget
end

@Target("annotation")
@Retention(model)
annotation Retention is
  required value : RetentionPolicy
end

@Target("annotation")
@Retention(model)
annotation Repeatable is
  optional value : Boolean = true
end

```

```

// Annotations communes
@Target("all")
@Retention(model)
annotation Experimental is
    optional reason : String = "Feature is experimental and may change"
end

@Target("all")
@Retention(model)
annotation Deprecated is
    required reason : String
    optional since : String = ""
    optional replacement : String = ""
end

@Target("all")
@Retention(model)
annotation Version is
    required value : String
end

@Target("all")
@Retention(model)
annotation Author is
    required name : String
    optional email : String = ""
    optional organization : String = ""
end

@Target("all")
@Retention(model)
annotation Scope is
    required level : ScopeLevel
    optional description : String = ""
end

@Target("all")
@Retention(model)
annotation Confidentiality is
    required level : ConfidentialityLevel
    optional classification : String = ""
    optional handling_instructions : String = ""
end

@Target("all")
@Retention(model)

```

```

annotation Documented is
  required summary : String
  optional description : String = ""
  optional examples : String = ""
  optional see_also : String = ""
end

```

meshr.types

```
// Types de base et communs
```

```

enum Status is (
  "active",
  "inactive",
  "pending",
  "suspended",
  "archived"
)

```

```

enum Priority is (
  "low",
  "medium",
  "high",
  "critical"
)

```

```

aspect ContactInfo is
  email : String
  phone : String
  mobile : String
  website : String
end

```

```

aspect Address is
  street : String
  city : String
  postal_code : String
  country : String
end

```

```

trait WithContactInfo is
  email : String
  phone : String

  aspects {
    ContactInfo {
      email: email,

```

```

        phone: phone,
        mobile: "",
        website: ""
    }
}
end

```

```

meshr.analytics
// Types pour l'analytics
enum MetricType is (
    "counter",
    "gauge",
    "histogram",
    "summary"
)

type relation DATA_LINEAGE is
    source_system : String
    transformation : String
    frequency : Interval
    last_updated : Timestamp
end

```

```

// Aspects d'analytics
aspect DataLineage is
    source_systems : List of String
    transformations : List of String
    refresh_frequency : Interval
    sla : Interval
end

```

Structure actuelle

```

stdlib/
  core/
    security.meshr      # Modules fondamentaux
    governance.meshr    # Aspects et types de sécurité
    lifecycle.meshr     # Gouvernance des données
    types.meshr         # Cycle de vie des données
    annotations.meshr   # Types de base et communs
    examples/           # Annotations standard et méta-annotations
    simple-customer.meshr
    customer-entity.meshr
    simple-imports.meshr
    imports-demo.meshr

```

```

        annotations-usage.meshr
        meta-annotations.meshr
business/          # Modules métier (à venir)
        domains.meshr
        entities.meshr
        processes.meshr
analytics/        # Modules d'analytics (à venir)
        metrics.meshr
        lineage.meshr
        quality.meshr
integrations/     # Intégrations réglementaires (à venir)
        gdpr.meshr
        sox.meshr
        iso27001.meshr

```

Cas d'usage

```

// Exemple d'utilisation de la stdlib
import { DataClassification, GDPRCompliance, WithSecurity } from meshr.security
import { DataStewardship, WithStewardship } from meshr.governance
import { WithLifecycle } from meshr.lifecycle
import { Version, Author, Documented, Confidentiality } from meshr.annotations

@Version("1.0.0")
@Author(name="Data Team", email="data@company.com")
@Documented(summary="Customer entity with full governance")
@Confidentiality(level="confidential", classification="PII")
entity Customer with WithSecurity, WithStewardship, WithLifecycle is
    customer_id : String
    email : String
    personal_data : String

    aspects {
        DataClassification {
            level: "confidential",
            encryption_required: true
        },
        GDPRCompliance {
            data_subject_rights: List["access", "rectification", "erasure"],
            retention_period: Interval 7 year,
            lawful_basis: "consent",
            dpo_contact: "dpo@company.com"
        }
    }
}
end

```


Roadmap

- ☒ **Phase 1** : Modules `security` et `governance`
 - ☒ **Phase 2** : Modules `lifecycle`, `types` et `annotations`
 - ☒ **Phase 3** : Exemples d'utilisation et documentation
 - ☐ **Phase 4** : Modules `business` et `analytics`
 - ☐ **Phase 5** : Intégrations réglementaires
 - ☐ **Phase 6** : Outils de validation et génération avancés
-

À venir

- Structuration des aspects (inline vs référence)
 - Système de types
 - Imports / Références croisées
 - Mécanisme de validation
 - Export YAML et JSON (Rego, Aspect (DataPlex Universal Catalog)...)
-

Bonnes pratiques et conventions

Cette section présente les bonnes pratiques recommandées pour écrire du code Meshr-lang efficace, maintenable et conforme aux standards de l'organisation.

Conventions de nommage

Modules

- **Format** : `domain.subdomain` (ex: `marketing.analytics`, `core.types`)
- **Convention** : Utiliser des noms en minuscules avec des points comme séparateurs
- **Éviter** : Les noms trop génériques comme `common`, `utils`, `shared`

Entités et types

- **Format** : `PascalCase` (ex: `Customer`, `ProductCatalog`, `OrderStatus`)
- **Convention** : Noms explicites qui reflètent le concept métier
- **Éviter** : Abréviations et acronymes non standardisés

Champs et propriétés

- **Format** : `camelCase` (ex: `customerId`, `emailAddress`, `createdAt`)
- **Convention** : Noms descriptifs et cohérents
- **Éviter** : Noms génériques comme `data`, `info`, `value`

Énumérations

- **Valeurs** : UPPER_SNAKE_CASE (ex: ACTIVE, PENDING_APPROVAL, HIGH_PRIORITY)
- **Convention** : Valeurs explicites et auto-documentées
- **Éviter** : Valeurs numériques ou abréviations

Organisation des modules

Structure recommandée

```
modules/  
  core/  
    types.meshr          # Types fondamentaux  
    governance.meshr     # Aspects de gouvernance  
    security.meshr       # Aspects de sécurité  
  business/  
    customer.meshr       # Domaine client  
    product.meshr        # Domaine produit  
    order.meshr          # Domaine commande  
  shared/  
    common.meshr         # Types partagés  
    policies.meshr       # Politiques transversales
```

Principes d'organisation

- **Un module = un domaine métier** : Éviter les modules trop larges
- **Dépendances claires** : Minimiser les dépendances circulaires
- **Séparation des responsabilités** : Séparer les types, aspects, et politiques

Utilisation des traits et aspects

Traits

- **Factorisation** : Créer des traits pour les patterns récurrents
- **Composition** : Préférer la composition à l'héritage complexe
- **Naming** : Préfixer par With (ex: WithAudit, WithTimestamps)

Aspects

- **Spécificité** : Créer des aspects spécialisés plutôt que génériques
- **Réutilisabilité** : Concevoir pour la réutilisation entre domaines
- **Documentation** : Toujours documenter le rôle et l'usage des aspects

Documentation et commentaires

Commentaires obligatoires

- **Modules** : Description du domaine et des responsabilités
- **Entités complexes** : Explication du rôle métier
- **Aspects personnalisés** : Usage et contraintes
- **Relations** : Cardinalité et règles métier

Exemple de documentation

```
/**
 * Module de gestion des clients et de leurs données personnelles.
 *
 * Ce module définit les entités centrales du domaine client,
 * incluant la conformité RGPD et les aspects de sécurité.
 */
module business.customer

// Entité représentant un client dans le système
entity Customer with WithAudit, WithSecurity is
  customerId: String pattern "^CUST-[0-9]{8}$"
  email: String pattern "^[^@]+@[^@]+$"
  // ... autres champs
end
```

Gestion des erreurs et validation

Contraintes de validation

- **Patterns** : Utiliser des expressions régulières pour valider les formats
- **Ranges** : Définir des plages de valeurs pour les nombres
- **Required fields** : Marquer explicitement les champs obligatoires

Gestion des versions

- **Versioning sémantique** : Suivre le format MAJOR.MINOR.PATCH
- **Rétrocompatibilité** : Éviter les breaking changes dans les versions mineures
- **Dépréciation** : Utiliser les annotations `@deprecated` pour les évolutions

Sécurité et conformité

Données sensibles

- **Classification** : Toujours classer les données avec des aspects appropriés
- **Chiffrement** : Spécifier les exigences de chiffrement
- **Accès** : Définir les niveaux d'accès et permissions

Conformité réglementaire

- **RGPD** : Inclure les aspects de rétention et de consentement
- **Audit** : Traçabilité des modifications et accès
- **Gouvernance** : Définir les rôles et responsabilités

Tests et validation

Tests de modèles

- **Validation syntaxique** : Vérifier la syntaxe des fichiers
- **Validation sémantique** : Tester les contraintes et relations
- **Tests d'intégration** : Valider les imports et dépendances

Outils recommandés

- **Linting** : Utiliser les outils de validation automatique
- **CI/CD** : Intégrer la validation dans les pipelines
- **Documentation** : Générer automatiquement la documentation

Annexes

Annexe A — Grammaire EBNF (référence)

```
(* =====
Meshr-Lang - Grammaire EBNF (alignée ANTLR)
Mise à jour: 2025-09-12
===== *)

compilation-unit    = module-decl, { import-decl }, { export-decl }, { top-level-decl }, EOF ;

module-decl         = { annotation }, "module", qualified-name ;

import-decl         = "import", import-items, "from", qualified-name ;
import-items        = "*"
                    | identifier
                    | "{", import-item, { ",", import-item }, "}" ;
import-item         = identifier ;

export-decl         = "export", ( export-items | exportable-decl ) ;
export-items        = "{", identifier, { ",", identifier }, "}" ;

top-level-decl      = enum-decl | annotation-decl | record-decl | trait-decl | aspect-decl ;
exportable-decl     = enum-decl | entity-decl | type-relation-decl | relation-decl ;

enum-decl           = { annotation }, [ "sealed" ], "enum", identifier, [ enum-signature ],
enum-signature      = "(", enum-attribute, { ",", enum-attribute }, ")" ;
enum-attribute      = identifier, ":", ( qualified-name | base-type ), [ "=", annotation-val
```

```

enum-values      = enum-value, { ",", enum-value } ;
enum-value       = (identifier | string-literal), [ "(", enum-value-args, ")" ] ;
enum-value-args  = enum-value-arg, { ",", enum-value-arg } ;
enum-value-arg   = identifier, "=", annotation-value ;

annotation-decl  = { annotation }, "annotation", identifier, "is", annotation-field, { annotation-field } ;
annotation-field = ( "required" | "optional" ), identifier, ":", ( qualified-name | base-type ) ;

annotation       = "@", identifier, [ "(", [ annotation-args ], ")" ] ;
annotation-args  = annotation-arg, { ",", annotation-arg } ;
annotation-arg   = annotation-arg-pair | annotation-value ;
annotation-arg-pair = identifier, "=", annotation-value ;
annotation-value = string-literal | number-literal | boolean-literal | qualified-name | base-type ;

base-type        = "Boolean" | string-type | "Integer" | "Float" | "Double"
                  | "Date" | "Datetime" | "Time" | "Timestamp"
                  | "Geography" | "Bytes" | "Json"
                  | "Interval" | "Range" ;

(* Type String avec contraintes optionnelles *)
string-type      = "String", [ string-constraints ] ;
string-constraints = string-constraint, { string-constraint } ;
string-constraint = "pattern", string-literal
                  | "length", signed-number, "..", signed-number ;

interval-literal = "Interval", interval-single-part
                  | "Interval", string-literal, "year", "to", "month"
                  | "Interval", string-literal, "year", "to", "day"
                  | "Interval", string-literal, "year", "to", "hour"
                  | "Interval", string-literal, "year", "to", "minute"
                  | "Interval", string-literal, "year", "to", "second"
                  | "Interval", string-literal, "month", "to", "day"
                  | "Interval", string-literal, "month", "to", "hour"
                  | "Interval", string-literal, "month", "to", "minute"
                  | "Interval", string-literal, "month", "to", "second"
                  | "Interval", string-literal, "day", "to", "hour"
                  | "Interval", string-literal, "day", "to", "minute"
                  | "Interval", string-literal, "day", "to", "second"
                  | "Interval", string-literal, "hour", "to", "minute"
                  | "Interval", string-literal, "hour", "to", "second"
                  | "Interval", string-literal, "minute", "to", "second" ;

interval-single-part = [ '-' ], number-literal, interval-unit ;
interval-unit        = "year" | "quarter" | "month" | "week" | "day"
                      | "hour" | "minute" | "second" | "millisecond" | "microsecond" ;

```

```

qualified-name      = identifier, { ".", identifier } ;
boolean-literal    = "true" | "false" ;

(* ===== RECORD DECLARATION ===== *)
record-decl        = "record", identifier, [ with-clause ], "is", record-field, { record-field } ;
record-field       = identifier, ":", type-ref, [ "=", annotation-value ], [ NEWLINE ] ;

(* ===== TRAIT DECLARATION ===== *)
trait-decl         = [ "sealed" ], "trait", identifier, [ with-clause ], "is", trait-field, { trait-field } ;
trait-field        = identifier, ":", type-ref, [ "=", annotation-value ], [ NEWLINE ] ;
with-clause        = "with", qualified-name, { ",", qualified-name } ;
trait-aspects      = "aspects", "{", aspect-instance, { ",", aspect-instance }, "}" ;

(* ===== ASPECT DECLARATION ===== *)
aspect-decl        = { annotation }, [ "abstract" ], "aspect", identifier, [ aspect-inheritance ], [ NEWLINE ] ;
aspect-inheritance = extends-clause, [ with-clause ]
                  | with-clause, [ extends-clause ] ;
extends-clause     = "extends", qualified-name ;
aspect-field       = identifier, ":", type-ref, [ "=", annotation-value ], [ NEWLINE ] ;

(* ===== ENTITY ===== *)
entity-decl        = { annotation }, [ "sealed" ], "entity", identifier, [ with-clause ], [ NEWLINE ] ;
entity-field       = identifier, ":", type-ref, [ "=", annotation-value ], [ NEWLINE ] ;
entity-aspects     = "aspects", "{", aspect-instance, { ",", aspect-instance }, "}" ;

(* ===== TYPE RELATION ===== *)
type-relation-decl = { annotation }, [ "sealed" ], "type", "relation", identifier, [ with-clause ], [ NEWLINE ] ;
type-relation-field = identifier, ":", type-ref, [ "=", annotation-value ], [ NEWLINE ] ;
type-relation-aspects = "aspects", "{", aspect-instance, { ",", aspect-instance }, "}" ;

(* ===== RELATION ===== *)
relation-decl      = { annotation }, [ "sealed" ], [ "bidirectional" ], "relation", identifier, [ with-clause ], [ NEWLINE ] ;
relation-type      = "of", "type", qualified-name ;
relation-endpoints = "from", relation-endpoint, "to", relation-endpoint ;
relation-endpoint  = qualified-name, "(", identifier, { ",", identifier }, ")" ;
relation-field-list = relation-field, { relation-field } ;
relation-field     = identifier, ":", type-ref, [ "=", annotation-value ], [ NEWLINE ] ;
relation-aspects   = "aspects", "{", aspect-instance, { ",", aspect-instance }, "}" ;

(* ===== ASPECT INSTANCES ===== *)
aspect-instance     = identifier, "{", [ aspect-instance-field-list ], "}" ;
aspect-instance-field-list = aspect-instance-field, { ",", aspect-instance-field } ;
aspect-instance-field = identifier, ":", annotation-value ;

(* ===== COLLECTION/COMPOSITE LITTÉRAUX ===== *)
list-literal        = "List", "[", [ annotation-value, { ",", annotation-value } ], "]" ;

```

```

map-literal      = "Map", "{", [ map-entry, { ",", map-entry } ], "}" ;
map-entry        = annotation-value, ":", annotation-value ;
record-literal   = "Record", "{", [ record-lit-field, { ",", record-lit-field } ], "}" ;
record-lit-field = identifier, ":", annotation-value ;
signed-number    = [ '-' ], number-literal ;
range-literal    = "Range", signed-number, "..", signed-number ;

(* ===== JSON LITTÉRAUX ===== *)
json-literal     = "Json", "{", json-object-content, "}"
                | "Json", "[", json-array-content, "]"
                | "Json", string-literal ;
json-object-content = json-pair, { ",", json-pair } ;
json-array-content  = json-value, { ",", json-value } ;
json-pair           = string-literal, ":", json-value ;
json-value          = string-literal
                | number-literal
                | boolean-literal
                | "null"
                | "{", json-object-content, "}"
                | "[", json-array-content, "]" ;

(* ===== GEOGRAPHY LITTÉRAUX ===== *)
geography-literal = "Geography", string-literal
                | "Geography", "{", json-object-content, "}" ;

(* ===== BYTES LITTÉRAUX ===== *)
bytes-literal     = "Bytes", string-literal
                | "Bytes", "[", bytes-array-content, "]"
                | "Bytes", "{", json-object-content, "}" ;
bytes-array-content = bytes-value, { ",", bytes-value } ;
bytes-value        = signed-number ;

string-literal    = "'", { character - "'" | '\\"' }, "'" ;
number-literal    = digit, { digit } | "0x", hex-digit, { hex-digit } ;
identifier        = letter, { letter | digit | "_" } ;
NEWLINE           = ("\r"?), "\n", { ("\r"?), "\n" } ;

```

Annexe B — Grammaire ANTLR (référence, Python target)

```

grammar MeshrModule;

// =====
// Entrée principale
// =====
compilationUnit
    : annotatedModuleDecl importDecl* exportDecl* topLevelDecl* EOF

```

```

;

// =====
// Déclarations de module
// =====
annotatedModuleDecl
  : annotation* 'module' qualifiedName
  ;

// =====
// Import / Export
// =====
importDecl
  : 'import' importItemsWithOptionalBraces 'from' qualifiedName
  ;

importItemsWithOptionalBraces
  : IDENTIFIER                                # SingleImport
  | '*'                                       # WildcardImport
  | '{' importItems '}'                     # GroupImport
  ;

importItems
  : IDENTIFIER (',' IDENTIFIER)+
  ;

exportDecl
  : 'export' exportableDecl                  # InlineExport
  | 'export' '{' exportItems '}'            # GroupedExport
  ;

exportItems
  : IDENTIFIER (',' IDENTIFIER)*
  ;

// =====
// Déclarations de haut niveau
// =====
topLevelDecl
  : annotatedEnumDecl
  | sealedEnumDecl
  | annotatedAnnotationDecl
  | sealedRecordDecl
  | recordDecl
  | sealedTraitDecl
  | traitDecl

```



```

    | annotatedAspectDecl
    | sealedAspectDecl
    | annotatedEntityDecl
    | sealedEntityDecl
    | annotatedTypeRelationDecl
    | sealedTypeRelationDecl
    | annotatedRelationDecl
    | sealedRelationDecl
    ;

// ===== ENUM =====
exportableDecl
    : enumDecl
    | sealedEnumDecl
    | entityDecl
    | sealedEntityDecl
    | typeRelationDecl
    | sealedTypeRelationDecl
    | relationDecl
    | sealedRelationDecl
    ;

annotatedEnumDecl
    : annotation* enumDecl
    ;

sealedEnumDecl
    : SEALED enumDecl
    ;

// ===== ENTITY =====
annotatedEntityDecl
    : annotation* entityDecl
    ;

sealedEntityDecl
    : SEALED entityDecl
    ;

enumDecl
    : 'enum' IDENTIFIER enumSignature? 'is' '(' enumValueList ')'
    ;

enumSignature
    : '(' enumAttributeList ')'
    ;

```

```

enumAttributeList
    : enumAttribute (',' enumAttribute)*
    ;

enumAttribute
    : IDENTIFIER ':' typeRef ('=' annotationValue)?
    ;

enumValueList
    : enumValue (',' enumValue)*
    ;

enumValue
    : (IDENTIFIER | STRING_LITERAL) ((' enumValueArgList? '))?
    ;

enumValueArgList
    : enumValueArg (',' enumValueArg)*
    ;

enumValueArg
    : IDENTIFIER '=' annotationValue
    ;

// ===== ENTITY DECLARATION =====
entityDecl
    : ENTITY IDENTIFIER withClause? 'is' entityFieldList entityAspects? 'end'
    ;

entityFieldList
    : entityField+
    ;

entityField
    : IDENTIFIER ':' typeRef ('=' annotationValue)?
    ;

entityAspects
    : ASPECTS '{' aspectInstanceList '}'
    ;

aspectInstanceList
    : aspectInstance (',' aspectInstance)*
    ;

```

```

aspectInstance
  : IDENTIFIER '{' aspectInstanceFieldList? '}'
  ;

aspectInstanceFieldList
  : aspectInstanceField (',' aspectInstanceField)*
  ;

aspectInstanceField
  : IDENTIFIER ':' annotationValue
  ;

// ===== TYPE RELATION DECLARATION =====
annotatedTypeRelationDecl
  : annotation* typeRelationDecl
  ;

sealedTypeRelationDecl
  : SEALED typeRelationDecl
  ;

typeRelationDecl
  : TYPE RELATION IDENTIFIER withClause? 'is' typeRelationFieldList typeRelationAspects?
  ;

typeRelationFieldList
  : typeRelationField+
  ;

typeRelationField
  : IDENTIFIER ':' typeRef ('=' annotationValue)?
  ;

typeRelationAspects
  : ASPECTS '{' aspectInstanceList '}'
  ;

// ===== RELATION DECLARATION =====
annotatedRelationDecl
  : annotation* relationDecl
  ;

sealedRelationDecl
  : SEALED relationDecl
  ;

```

```

relationDecl
    : BIDIRECTIONAL? RELATION IDENTIFIER relationType? withClause? 'is' relationEndpoints re
    ;

relationType
    : 'of' TYPE IDENTIFIER
    ;

relationEndpoints
    : FROM relationEndpoint TO relationEndpoint
    ;

relationEndpoint
    : IDENTIFIER '(' IDENTIFIER (',' IDENTIFIER)* ')'
    ;

relationFieldList
    : relationField+
    ;

relationField
    : IDENTIFIER ':' typeRef ('=' annotationValue)?
    ;

relationAspects
    : ASPECTS '{' aspectInstanceList '}'
    ;

// ===== ANNOTATION DECLARATION =====
annotatedAnnotationDecl
    : annotation* annotationDecl
    ;

annotationDecl
    : 'annotation' IDENTIFIER 'is' annotationFieldList 'end'
    ;

annotationFieldList
    : annotationField+
    ;

annotationField
    : ('required' | 'optional') IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
    ;

// ===== ANNOTATION USAGE =====

```

```

annotation
    : '@' IDENTIFIER ('(' annotationArgs? ')')?
    ;

annotationArgs
    : annotationArg (',' annotationArg)*
    ;

annotationArg
    : annotationArgPair
    | annotationValue
    ;

annotationArgPair
    : IDENTIFIER '=' annotationValue
    ;

// ===== TYPES DE BASE =====

// Types de base reconnus
baseType
    : 'Boolean'
    | stringType
    | 'Integer'
    | 'Float'
    | 'Double'
    | 'Date'
    | 'Datetime'
    | 'Time'
    | 'Geography'
    | 'Timestamp'
    | 'Bytes'
    | 'Json'
    | 'Interval'
    | 'Range'
    ;

// Type String avec contraintes optionnelles
stringType
    : 'String' stringConstraints?
    ;

stringConstraints
    : stringConstraint (stringConstraint)*
    ;

```

```

stringConstraint
  : 'pattern' STRING_LITERAL
  | 'length' signedNumber '..' signedNumber
  ;

// ===== SHARED =====
qualifiedName
  : IDENTIFIER ('.' IDENTIFIER)*
  ;

// ===== TYPE REFERENCES =====
typeRef
  : qualifiedName
  | baseType
  | listType
  | mapType
  | rangeType
  ;

listType
  : 'List' 'of' typeRef
  ;

mapType
  : 'Map' 'of' typeRef 'to' typeRef
  ;

rangeType
  : 'Range' 'of' typeRef
  ;

// ===== ANNOTATION USAGE =====
annotationValue
  : STRING_LITERAL
  | NUMBER_LITERAL
  | BOOLEAN_LITERAL
  | qualifiedName
  | intervalLiteral
  | listLiteral
  | mapLiteral
  | recordLiteral
  | rangeLiteral
  | jsonLiteral
  | geographyLiteral
  | bytesLiteral
  ;

```

```

// ===== INTERVAL LITERALS =====
intervalLiteral
  : 'Interval' intervalSinglePart
  | 'Interval' STRING_LITERAL 'year' 'to' 'month'
  | 'Interval' STRING_LITERAL 'year' 'to' 'day'
  | 'Interval' STRING_LITERAL 'year' 'to' 'hour'
  | 'Interval' STRING_LITERAL 'year' 'to' 'minute'
  | 'Interval' STRING_LITERAL 'year' 'to' 'second'
  | 'Interval' STRING_LITERAL 'month' 'to' 'day'
  | 'Interval' STRING_LITERAL 'month' 'to' 'hour'
  | 'Interval' STRING_LITERAL 'month' 'to' 'minute'
  | 'Interval' STRING_LITERAL 'month' 'to' 'second'
  | 'Interval' STRING_LITERAL 'day' 'to' 'hour'
  | 'Interval' STRING_LITERAL 'day' 'to' 'minute'
  | 'Interval' STRING_LITERAL 'day' 'to' 'second'
  | 'Interval' STRING_LITERAL 'hour' 'to' 'minute'
  | 'Interval' STRING_LITERAL 'hour' 'to' 'second'
  | 'Interval' STRING_LITERAL 'minute' 'to' 'second'
  ;

intervalSinglePart
  : '-'? NUMBER_LITERAL intervalUnit
  ;

intervalUnit
  : 'year' | 'quarter' | 'month' | 'week' | 'day'
  | 'hour' | 'minute' | 'second' | 'millisecond' | 'microsecond'
  ;

// ===== RECORD DECLARATION =====
recordDecl
  : 'record' IDENTIFIER withClause? 'is' recordFieldList 'end'
  ;

sealedRecordDecl
  : SEALED recordDecl
  ;

recordFieldList
  : recordField+
  ;

recordField
  : IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
  ;

```

```

// ===== TRAIT DECLARATION =====
traitDecl
  : 'trait' IDENTIFIER withClause? 'is' traitFieldList traitAspects? 'end'
  ;

sealedTraitDecl
  : SEALED traitDecl
  ;

traitFieldList
  : traitField+
  ;

traitField
  : IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
  ;

traitAspects
  : ASPECTS '{' aspectInstanceList '}'
  ;

withClause
  : 'with' qualifiedName (',' qualifiedName)*
  ;

// ===== ASPECT DECLARATION =====
annotatedAspectDecl
  : annotation* aspectDecl
  ;

aspectDecl
  : 'abstract'? 'aspect' IDENTIFIER aspectInheritance? 'is' aspectFieldList 'end'
  ;

sealedAspectDecl
  : SEALED aspectDecl
  ;

aspectInheritance
  : extendsClause withClause?
  | withClause extendsClause?
  ;

extendsClause
  : 'extends' qualifiedName

```



```

        ;

aspectFieldList
    : aspectField+
    ;

aspectField
    : IDENTIFIER ':' typeRef ('=' annotationValue)? NEWLINE?
    ;

// ===== COLLECTION AND COMPOSITE LITERALS =====
listLiteral
    : 'List' '[' (annotationValue (',' annotationValue)*)? ']'
    ;

mapLiteral
    : 'Map' '{' (mapEntry (',' mapEntry)*)? '}'
    ;

mapEntry
    : annotationValue ':' annotationValue
    ;

recordLiteral
    : 'Record' '{' (recordLitField (',' recordLitField)*)? '}'
    ;

recordLitField
    : IDENTIFIER ':' annotationValue
    ;

rangeLiteral
    : 'Range' signedNumber '..' signedNumber
    ;

// ===== JSON LITERALS =====
jsonLiteral
    : 'Json' '{' jsonObjectContent '}'
    | 'Json' '[' jsonArrayContent ']'
    | 'Json' STRING_LITERAL
    ;

jsonObjectContent
    : jsonPair (',' jsonPair)*
    | // empty
    ;

```

```

jsonArrayContent
    : jsonValue (',' jsonValue)*
    | // empty
    ;

jsonPair
    : STRING_LITERAL ':' jsonValue
    ;

jsonValue
    : STRING_LITERAL
    | NUMBER_LITERAL
    | BOOLEAN_LITERAL
    | 'null'
    | '{' jsonObjectContent '}'
    | '[' jsonArrayContent ']'
    ;

signedNumber
    : '-'? NUMBER_LITERAL
    ;

// ===== GEOGRAPHY LITERALS =====
geographyLiteral
    : 'Geography' STRING_LITERAL
    | 'Geography' '{' jsonObjectContent '}'
    ;

// ===== BYTES LITERALS =====
bytesLiteral
    : 'Bytes' STRING_LITERAL
    | 'Bytes' '[' byteArrayContent ']'
    | 'Bytes' '{' jsonObjectContent '}'
    ;

byteArrayContent
    : bytesValue (',' bytesValue)*
    | // empty
    ;

bytesValue
    : signedNumber
    ;

// ===== TERMINALS =====

```

```

fragment ESC
    : '\\' ["\\/\bfnrt]
    ;

AT: '@';
STRING_LITERAL: '"' (ESC | ~["\\r\n])* '"';

// Keywords
SEALED: 'sealed';
ENTITY: 'entity';
TYPE: 'type';
RELATION: 'relation';
BIDIRECTIONAL: 'bidirectional';
FROM: 'from';
TO: 'to';
ASPECTS: 'aspects';

NUMBER_LITERAL
    : [0-9]+ ('.' [0-9]+)?
    | '0' [xX] [0-9a-fA-F]+
    ;

BOOLEAN_LITERAL
    : 'true' | 'false'
    ;

IDENTIFIER: [a-zA-Z_][a-zA-Z0-9_]* ;

WS: [ \t\r\n]+ -> skip ;
NEWLINE: ('\r'? '\n')+ -> skip ;
COMMENT: '//' ~[\r\n]* -> skip ;
MULTILINE_COMMENT: '/*' .*? '*/' -> skip ;

```